ErrSight

Interview Guide

Everything you need to know to explain and defend this project: what it does, how it works, why each decision was made, what is weak, and what you will likely be asked.

> **How to use this guide**
>
> **Read order for interview prep:** Sections 1-3 (big picture) → 6-8 (the parts you must be able to explain deeply) → 16 (limitations, say these before they find them) → 17 (Q&A) → 19 (demo script).
>
> **Golden rule:** be honest. Know your limitations list cold. An interviewer respects "I know that is a weakness, here is how I would fix it" far more than a bluff that gets caught.

Contents

1 Project at a glance
2 The problem it solves
3 Architecture & end-to-end flow
4 Tech stack & why
5 Database design
6 Ingestion (fast-ack)
7 The worker (core logic)
8 Auth & security
9 Dashboard API & pagination
10 JavaScript SDK
11 React dashboard
12 Docker, CI & testing
13 Concepts glossary
14 Design decisions & tradeoffs
15 Bugs I found & fixed
16 Known limitations
17 Interview Q&A
18 Resume points
19 3-minute demo script
20 Cheat sheet & pre-interview checklist

1 Project at a glance

- **What it is:** a Sentry-lite error tracking platform. A developer installs a small JS SDK; when their app crashes, the error appears on a dashboard, **grouped by root cause**, with an alert when a **new** bug shows up.
- **Three packages in one repo:** backend (API + worker), `dashboard/` (React), `sdk/javascript/` (the SDK).
- **Size:** 6 database tables, 6 migrations, 5 Docker services, 3 CI jobs, ~36 unit tests.
- **Strongest parts (lead with these):** fast-ack ingestion, idempotent transaction-safe worker, fingerprint grouping, refresh-token rotation.

> **Your 30-second pitch (memorize this)**
>
> ErrSight is a mini Sentry. You drop an SDK into your app, and when it crashes the error is captured and sent to my API, which acknowledges in under 50 ms and pushes it onto a Redis queue. A background worker scrubs PII, fingerprints the error so 10,000 crashes of the same bug become one group, and stores it in Postgres, alerting on new groups. A React dashboard shows the grouped errors with context. The interesting engineering is keeping ingestion fast during spikes, and making the worker idempotent so retries never double-count.

2 The problem it solves

- **Users hit errors, developers find out too late** (or never).
- **Noise:** 10,000 crash logs can be the same single bug. Without grouping, nobody can find the signal.
- **No context in raw logs:** which release broke it, which environment, how many users, what the user did just before.
- **ErrSight answer:** capture automatically, group by fingerprint, store context (release, environment, breadcrumbs, user count), alert on new groups.

3 Architecture & end-to-end flow



Step by step

- **1.** App throws. SDK captures it (browser `error` / `unhandledrejection`, Node `uncaughtException` / `unhandledRejection`) and attaches the last 20 breadcrumbs.
- **2.** SDK sends `POST /api/ingest` with the project `x-api-key`. Fire-and-forget; never blocks the host app.
- **3.** API: find project by key → Redis rate limit (100/min per project) → zod validation → `queue.add()` → respond `200 {eventId}`.
- **4.** BullMQ holds the job. If processing fails it retries (3 attempts, backoff 1s → 2s → 4s).
- **5.** Worker (5 concurrent) scrubs PII from message, stack and context.

- **6.** Worker computes the fingerprint, opens ONE DB transaction: find/create group, insert event (idempotent), bump counters.
- **7.** If the group is brand new: send an alert (Slack webhook if configured; always logged).
- **8.** Dashboard (JWT) lists groups, shows events, lets the user resolve a group.

> **The one idea behind the whole design**
> The request path never writes to Postgres. That is why the API stays fast when thousands of clients crash at the same time: the queue absorbs the spike and the worker drains it at a controlled rate.

4 Tech stack & why

Tech	Why it was chosen
Node.js + Express 5 + TypeScript	Express 4 is nearing end-of-life; 5 is the current recommended release. TypeScript catches shape mismatches between models, routes, worker.
PostgreSQL (raw SQL via `pg`)	Full control over queries (atomic counters, `ON CONFLICT`, row-comparison cursors). No ORM hiding what runs. Shows SQL skill.
Redis + BullMQ	Queue between ingestion and processing (spike protection, retries, backoff). Redis also backs the rate limiter.
zod 4	Runtime validation of request bodies; also the place payload size caps live.
JWT (jsonwebtoken) + opaque refresh tokens	Stateless short-lived access tokens; revocable DB-backed refresh tokens.
bcryptjs	Slow salted password hashing. Pure JS = no native build step in Alpine Docker (tradeoff: slower than native bcrypt).
winston	Structured logging (JSON in production).
helmet + cors	Secure default headers; per-route CORS policy.
Vitest	Fast, TypeScript-native test runner with near-zero config.
React 18 + Vite 8 + React Router 6 + axios	Simple SPA; axios interceptors handle token attach + refresh-on-401.
Docker + docker compose	One command runs Postgres, Redis, migrations, API, worker. Multi-stage Dockerfile; production runs compiled JS.
GitHub Actions	Typecheck + test + build on every push for all 3 packages.

5 Database design

Table	Key columns & constraints
users	`email` UNIQUE, `password_hash`, `email_verified_at` (nullable)
projects	`api_key` UNIQUE (uuid v4), `owner_id` FK → users ON DELETE CASCADE, `platform`
error_groups	`UNIQUE(project_id, fingerprint)` (the grouping guarantee), `status` (unresolved/resolved/ignored), `level`, `event_count`, `user_count`, `first_seen`, `last_seen`
error_events	`event_id` UNIQUE (idempotency key), FK to group, `message`, `stack_trace`, `release`, `environment`, `user_id_hash`, `breadcrumbs` JSONB, `context` JSONB
refresh_tokens	`token_hash` UNIQUE (sha256), `expires_at`, `revoked_at`
password_reset_tokens	`token_hash` UNIQUE, `expires_at`, `used_at` (one-time use)

Migrations (run in order by `scripts/migrate.ts`, all re-runnable)

#	What it does
001	Core tables (users, projects, error_groups, error_events) + indexes
002	`level` column on error_groups (so groups can be filtered by level)
003	`event_id` UNIQUE on error_events (idempotency key)
004	Composite index `(project_id, last_seen DESC, id DESC)` for cursor pagination
005	`refresh_tokens` table
006	`email_verified_at` on users + `password_reset_tokens` table

Design notes worth saying out loud

- **UUID primary keys** (`gen_random_uuid()`): safe to expose in URLs, no guessable sequence.
- **Denormalized counters** (`event_count`, `user_count`, `level`) on the group: listing groups is ONE query, no join/aggregate per row (avoids N+1).
- **JSONB** for breadcrumbs/context: variable shape, always read with the event, never queried independently.
- **Hashed tokens in DB:** refresh/reset tokens are stored as sha256 hashes, like passwords; a DB leak does not hand out usable tokens.
- **FK indexes on purpose:** Postgres does NOT auto-index foreign keys; without them joins and cascade deletes scan the table.
- **Migrations idempotent** (`IF NOT EXISTS`); inline `UNIQUE` in `ADD COLUMN IF NOT EXISTS` because Postgres has no `ADD CONSTRAINT IF NOT EXISTS`.

6 Ingestion (fast-ack)

- **Public route**, authenticated by `x-api-key` (project key), not JWT: the SDK runs inside customer apps where no user is logged in.
- **Pipeline order matters:** API key lookup → rate limit → body validation → queue push → 200.
- **Rate limit:** 100 events/min per project, Redis `INCR` + `PEXPIRE`. Over limit gives `429` + `Retry-After`.
- **Payload caps (zod):** message ≤ 1000 chars, stack ≤ 5000, breadcrumb message ≤ 500, ≤ 20 breadcrumbs.
- **Queue options:** `attempts: 3`, exponential backoff starting at 1s (1s, 2s, 4s), `removeOnComplete: 100`, `removeOnFail: 500` (failed jobs kept for inspection = dead-letter style).
- `eventId` **(UUID)** is generated here and travels with the job; it becomes the idempotency key in the DB.
- **If Redis is down:** `queue.add` throws, API returns `500` fast (does not hang). The event is lost for that request (known limitation).
- **CORS is deliberately open on this route:** SDKs run on arbitrary third-party sites; restricting it would break the product.

> **Why fast-ack / why a queue**
>
> If the API wrote to Postgres on every request, a crash storm (everyone hits the same bug) would exhaust DB connections and slow the very API that is supposed to record the crash. Doing only a Redis push keeps the response in a few ms; the worker absorbs the load at its own pace.

7 The worker (core logic)

Processing order

- **1. Scrub PII** from message, stack, and context (before anything is stored).
- **2. Fingerprint** = hash of normalized message + stable stack frame.
- **3. One DB transaction:** find-or-create group → insert event with `ON CONFLICT (event_id) DO NOTHING` → if inserted: bump `event_count` + recompute `user_count`.
- **4. After commit:** if the group was newly created, `sendAlert()` (never on repeat events).
- **5. Errors are thrown, not swallowed** so BullMQ retries. `completed` / `failed` events are logged.

Fingerprinting

- **Normalize message:** quoted strings → `*`, emails → `*`, URLs → `*`, numbers → `*`, then lowercase. (Order matters: digits are inside emails/quotes, so strip those first.)
- **Key frame:** first stack frame NOT in `node_modules` / node internals, as `funcName@fileName`. **No line numbers.**
- **Hash:** SHA-256 of `normalizedMessage|keyFrame`, first 16 hex chars.

```
"user 42 not found"  --normalize-->  "user * not found"  \
"user 99 not found"  --normalize-->  "user * not found"   >  same fingerprint = SAME group
(same stack frame: findUser@user.js)                     /
```

- **Why strip line numbers:** they shift on almost every commit even for the same bug; including them would split one bug into many groups after each deploy.
- **Why 16 hex chars:** we need a short stable grouping key, not cryptographic strength. 64 bits is plenty to avoid accidental collisions at this scale.

Idempotency (the strongest talking point)

- **Problem:** BullMQ is **at-least-once**: a job can be delivered twice (e.g. worker crashes after commit, before ack). Naive code would insert twice and double-count.
- **Fix:** every event carries a unique `event_id`. The insert is `ON CONFLICT (event_id) DO NOTHING`. If nothing was inserted (conflict) the transaction is a no-op: no counter bump, no alert.
- **Counters are bumped only after a successful insert**, inside the same transaction. Crash anywhere = full rollback, retry is safe.
- **Race on new group:** two workers can both see "no group". `UNIQUE(project_id, fingerprint)` + `ON CONFLICT DO NOTHING` returns null for the loser, who just re-fetches the winner's row.
- **Why not catch the unique-violation error?** In Postgres, any error inside a transaction aborts it; every later query fails until ROLLBACK. `ON CONFLICT` avoids raising an error at all.
- **Models accept an optional** `DbExecutor` so the same method runs on the pool or inside a transaction (otherwise a model call inside `withTransaction` would silently use a different connection).
- `user_count` **is recomputed** (`COUNT(DISTINCT user_id_hash)`), not incremented: distinct counts are not additive.

PII scrubbing

- `scrubPII` **(free text):** password fields, JWTs, emails, credit-card-like digit runs, long (32+ char) key-like strings → `[REDACTED]`. Specific patterns run before the generic long-key catch-all.
- `scrubContext` **(structured):** redacts keys `email`, `password`, `token`, `apiKey` (case-insensitive) **at any nesting depth**, and scrubs every string value.
- **Honest caveat:** regex heuristics can miss unusual PII; it reduces risk, it is not a guarantee.
- **User identifiers** are stored only as a sha256 hash (`user_id_hash`).

8 Auth & security

Two separate auth models

- **Humans (dashboard):** JWT access token (15 min) + rotating refresh token (30 days).
- **Machines (SDK):** project API key in `x-api-key`. Can be regenerated from the dashboard (old key dies immediately).

Login / refresh flow

- **Login:** bcrypt compare → issue access JWT + random 32-byte refresh token. Only the sha256 **hash** of the refresh token is stored.
- **Refresh with rotation:** presenting a refresh token revokes it and issues a brand-new pair. A stolen token that gets used makes the real owner's next refresh fail (a visible signal).
- **Logout:** revokes the refresh token server-side (works even if the access token already expired).
- **Dashboard interceptor:** on 401 it refreshes once and retries; concurrent 401s are **queued** behind a single refresh (otherwise parallel refreshes race against rotation and fail).

Account flows

- **Register:** 409 on duplicate email; bcrypt cost 10; generates a verification link.
- **Email verification:** signed JWT with `purpose: verify-email`, 24h. Login is NOT blocked by unverified status (design choice).
- **Forgot password:** always returns the same message whether or not the email exists (prevents email enumeration).
- **Reset password:** token is DB-stored, 1 hour, **single-use** (`used_at`). A reset also **revokes all refresh tokens** (assume compromise).
- **Email sending is a stub:** links are logged, not emailed (see limitations).

Abuse & hardening

- **Rate limiting:** ingest 100/min per project; auth routes 10 per 15 min per IP (brute force). Redis-backed, so it works across multiple API instances; **fails open** if Redis is down (logged loudly).
- `trust proxy` **= 1** so `req.ip` is the real client behind Docker/reverse proxy (otherwise everyone shares one bucket).
- **Ownership checks** on every project/group route; returns **404 not 403** so you cannot probe which IDs exist.
- **CORS split:** dashboard routes restricted to `CORS_ORIGIN` (closed by default in production); ingest open on purpose.
- `JWT_SECRET` **has no fallback in production** (app refuses to start; compose also refuses). Dev gets a default for convenience.
- **helmet**, 1 MB body limit, zod validation everywhere, no `dangerouslySetInnerHTML` (React escapes output, so no stored XSS from error text).
- **CSRF:** not applicable because auth is a Bearer header, not a cookie.

9 Dashboard API & pagination

Method	Path	Auth	Purpose
POST	/api/auth/register	none	Create account (logs verification link)
POST	/api/auth/login	none	Returns accessToken + refreshToken
POST	/api/auth/refresh	none	Rotate refresh token, new access token
POST	/api/auth/logout	none	Revoke refresh token
POST	/api/auth/verify-email	none	Verify email via token
POST	/api/auth/forgot-password	none	Request reset link (generic response)

Method	Path	Auth	Purpose
POST	/api/auth/reset-password	none	Set new password, revoke all sessions
GET/POST	/api/projects	JWT	List own projects / create (generates API key)
GET/DELETE	/api/projects/:id	JWT	One project / delete
POST	/api/projects/:id/regenerate-key	JWT	New API key, old one invalid
GET	/api/projects/:id/errors	JWT	Error groups (`status`, `level`, `limit`, `cursor`)
GET	/api/errors/:groupId	JWT	Group + last 20 events
POST	/api/errors/:groupId/resolve	JWT	Mark group resolved
POST	/api/ingest	x-api-key	SDK sends an event (fast-ack)
GET	/health	none	Checks Postgres + Redis, 503 if degraded

Cursor (keyset) pagination

- **Why not `OFFSET`:** `OFFSET 50000` makes Postgres scan and throw away 50,000 rows every request; cost grows with depth.
- **What we do:** order by `(last_seen DESC, id DESC)`; the cursor is the last row's `(last_seen, id)`, base64-encoded. Next page: `WHERE (last_seen, id) < (cursor)`.
- **Why `id` too:** `last_seen` is not unique; `id` is the tiebreaker so rows are never skipped/duplicated at a page boundary.
- **Response:** `{ groups, limit, nextCursor }`. A full page returns a cursor; a partial page returns `null` (end).
- **Index:** `(project_id, last_seen DESC, id DESC)` matches the query exactly.
- **Filtering** (`status`, `level`) happens in SQL `WHERE`, never in JavaScript after fetching.

10 JavaScript SDK

- **API:** `ErrSight.init({ apiKey, environment, release, userId, beforeSend })`, `captureException(err, context)`, `addBreadcrumb(message, data)`.
- **Auto-capture:** browser `error` + `unhandledrejection`; Node `uncaughtException` + `unhandledRejection`.
- **Singleton:** one shared instance per app. Error reporting is global; two clients would attach duplicate global listeners.
- **Breadcrumbs:** rolling log of the last 20 user actions. The stack says WHERE it broke, breadcrumbs say HOW the user got there.
- `beforeSend` **hook:** lets the app modify or drop (return null) an event before it leaves.
- **Fire-and-forget:** `fetch` is not awaited by user code. Reporting an error must never make the host app wait.
- **Never crashes the host:** whole `captureException` is in try/catch; network errors are `.catch()`ed; `JSON.stringify` failures (circular context) are caught.
- **Truncation:** message 1000, stack 5000, breadcrumb 500, matching server caps. Without it an oversized event would be rejected with 400 and silently lost.
- **Zero runtime dependencies** (native `fetch`, Node 18+). Own `package.json` and `tsconfig`.

11 React dashboard

- **Pages:** Login, Forgot/Reset Password, Verify Email, Dashboard, Legal (privacy/terms template).
- **Dashboard layout:** project selector, API key panel (masked, show/copy/regenerate with confirmation), error list, error detail with Resolve button.
- **State flow:** `Dashboard` owns `selectedProjectId` / `selectedGroupId`; children fetch their own data from props via `useEffect`. Data flows down as props, events flow up as callbacks.
- **Resolve:** POST, then update local state from the server response (no reload), and bump a `refreshKey` so the list refetches.
- **Error handling:** every API call has a `catch`; failed loads show an error + Retry button, distinct from the "All clear" empty state (this was a real bug; see section 15).
- **Tokens in localStorage** (access + refresh): simple, but readable by any XSS. Know this tradeoff.
- **Accessibility:** aria-labels on unlabeled controls, error rows are keyboard-operable (`role=button`, Enter/Space), visible `:focus-visible`, badge colors fixed to meet WCAG AA contrast (warning amber was 3.2:1, now ~5:1).
- **Not built:** pagination/filter UI (backend supports it), live updates (data loads on open/refresh, not real-time).

12 Docker, CI & testing

Docker

- **5 services:** `postgres`, `redis`, `migrate` (one-off), `api`, `worker`. API/worker wait on healthy DB/Redis AND successful migration (`service_completed_successfully`).
- **Multi-stage Dockerfile:** build stage compiles TypeScript; production stage installs prod deps only (`npm ci --omit=dev`) and runs compiled `dist/` with plain `node` (not ts-node).
- `migrate` **uses the build stage** because it needs `tsx` (a dev dependency) to run `scripts/migrate.ts`.
- `npm ci` needs committed `package-lock.json` files (reproducible installs).
- **Graceful shutdown:** API handles SIGTERM (stop accepting, close DB pool + Redis); worker calls `worker.close()` to finish active jobs.
- `/health` really checks Postgres + Redis (503 when degraded), so orchestrators can act on it.

CI (GitHub Actions)

- 3 parallel jobs: backend (typecheck src + scripts, test, build, audit), dashboard (typecheck, build), SDK (typecheck, build).

Testing: what exists vs what does not

- **Exists (~36 Vitest unit tests):** fingerprint (normalization, key-frame extraction, grouping behavior), PII scrubbing (incl. nested objects/arrays), rate limiter (limit, per-key isolation, window reset, fail-open).
- **Rate limiter tests inject a fake Redis client** (dependency injection) instead of mocking the module.
- **Smoke script:** `npm run test:ingest` sends 5 events through the real API and checks that this run created exactly 2 new groups (measures the delta, so seeded data does not break it).
- **Does NOT exist:** integration tests against a real DB (the worker transaction/idempotency logic is untested automatically), frontend tests, load tests. Say this proactively.

13 Concepts glossary

Term	Meaning (in this project)
Fast-ack	Respond immediately after queuing; do slow work later in a worker.
Queue / BullMQ	Buffer between producer (API) and consumer (worker); gives retries and backoff.
Exponential backoff	Wait 1s, 2s, 4s between retries (`delay * 2^(attempt-1)`) so a struggling DB gets time to recover.
Dead-letter (failed set)	Jobs that exhausted retries are kept (`removeOnFail: 500`) for inspection instead of vanishing.
At-least-once delivery	A job may be delivered more than once; consumers must be idempotent.
Idempotency	Doing the same operation twice has the same effect as once (here: unique `event_id` + `ON CONFLICT`).
Fingerprint	Short hash identifying "the same bug" from normalized message + stable stack frame.
Transaction	Group of DB statements that all succeed or all roll back.
`ON CONFLICT DO NOTHING`	Insert that silently skips on unique violation instead of erroring (important inside transactions).
JWT	`header.payload.signature`; payload is readable (not encrypted), signature proves it was not tampered with.
Refresh-token rotation	Each refresh invalidates the old token and issues a new one; theft becomes detectable.
Keyset / cursor pagination	Page by "rows after this (last_seen, id)" instead of skipping N rows.
N+1 problem	1 query for a list + N queries for per-item data; avoided via denormalized counters.
Breadcrumbs	Recent user actions leading up to the error.
Fail open / fail closed	If a safety dependency (Redis rate limiter) breaks: allow requests (open) vs block (closed). We chose open.
Singleton	Exactly one shared instance (the SDK client).
Enumeration attack	Probing which emails/IDs exist from differing responses; avoided with generic responses and 404s.

14 Design decisions & tradeoffs

Decision	Why	Tradeoff / cost
Queue + worker, not direct DB insert	Fast API during spikes, retries, decoupling	More moving parts (Redis, worker); event lost if Redis is down at ingest
Raw SQL, no ORM	Control, atomic SQL, easy to explain	More boilerplate; dynamic WHERE built by hand
Denormalized counters/level on group	One-query listing, filter by level	Must keep counters in sync (done in the worker transaction)
Idempotency via unique event_id	Safe retries under at-least-once	Extra column + index
Refresh tokens with rotation	Short-lived access token, revocable sessions	More code, extra DB table, client interceptor complexity
Tokens in localStorage	Simple SPA integration	XSS can steal them (httpOnly cookie + CSRF is the alternative)
Redis rate limiter, fixed window	Shared across instances, auto-expiring	Allows 2x burst at window boundary (sliding window / token bucket fixes)

Decision	Why	Tradeoff / cost
Rate limiter fails open	Redis hiccup should not take login down	Brief unthrottled window during an outage
Ingest CORS open, dashboard CORS closed	SDK runs on any origin; dashboard has one known origin	Ingest is protected by API key + rate limit instead
bcryptjs (pure JS)	No native compile in Alpine	Slower than native bcrypt
Login not blocked by unverified email	Smoother demo/UX	Verification currently gates nothing
Unit tests only	Fast, deterministic, covers core logic	Worker/DB behavior not covered automatically

15 Bugs I found & fixed (good "challenge" stories)

- Interviewers love "tell me about a bug". Pick 2-3 of these and be able to explain cause and fix.

Bug	Root cause	Fix
Retried jobs could duplicate events / inflate counts	Group counter bumped before the event insert, outside any transaction; no idempotency key	`event_id` unique + `ON CONFLICT`, everything in one transaction, bump only after insert
Concurrent workers crashing on new groups	Catching the unique-violation error inside a transaction poisons it (Postgres aborts the txn)	`ON CONFLICT DO NOTHING` on group insert, then re-fetch
Same error x3 gave `event_count` 2	New group inserted with count 0; first event never counted	Bump counter after insert uniformly
`user_count` **was always 0**	Column existed, worker never wrote it	Recompute distinct users per event with a user id
PII leaked in nested context	`scrubContext` only checked top-level keys	Recursive `scrubValue` over objects/arrays
`release` **/ breadcrumbs never saved**	Worker hardcoded `breadcrumbs: []` and never passed release	Extract from context, validate in zod
Login had no brute-force protection	Rate limiter was only wired to ingestion	Generalized limiter factory, applied to auth routes
My own rate limiter leaked memory	In-memory Map never evicted stale keys	Moved to Redis with TTL (also fixes multi-instance)
Dashboard showed "All clear" when the API failed	`try/finally` with no `catch` in 5 handlers	Error state + Retry button everywhere
SDK silently dropped big errors	No client-side truncation; server 400s oversized payloads	Truncate to the same caps
`tsc` **failed: file outside** `rootDir`	scripts/ included under `rootDir: ./src`	Separate `scripts/tsconfig.json`
`docker compose up` **would crash at migrate**	Migrate service set production but no `JWT_SECRET`; config throws	Pass `JWT_SECRET` to the migrate service
test-ingest failed after seeding	Asserted an absolute group count	Measure delta before/after
Warning badge failed contrast	White on #d97706 = 3.2:1 (needs 4.5)	Darker #b45309 (~5:1)

16 Known limitations (say these BEFORE they find them)

> **Why this matters**
> Naming your own weaknesses with a fix ready is a senior signal. Never claim "production-ready", "highly scalable" or "fully tested".

- **Email delivery is a stub:** verification/reset links are logged, not emailed. Fix: plug in SES/SendGrid at the two `logger.info` calls.
- **Tests are unit-level only:** no integration test of the worker + DB, no frontend tests, no load testing. Fix: Vitest + a real test Postgres in CI.
- **Event loss if Redis is down at ingest:** API returns 500, SDK does not retry. Fix: SDK retry with backoff / local buffer, or a fallback path.
- **Tokens in localStorage:** XSS exposure. Fix: httpOnly cookie for the refresh token + CSRF protection.
- **Fixed-window rate limit:** can allow a 2x burst around the window edge. Fix: sliding window or token bucket.
- **Hot-row contention:** thousands of events for one group all `UPDATE` the same row, serializing. Fix: batch counter updates or aggregate in the worker.
- **No data retention policy:** `error_events` grows forever. Fix: time-partitioning and a cleanup job.
- **Alerting is basic:** one global Slack webhook, no per-project config or UI.
- **Dashboard is not real-time** and has no pagination/filter UI (API supports both).
- **No API versioning** (`/api/v1`), no OpenAPI spec, no metrics/tracing.
- **PII scrubbing is regex-based:** best-effort.
- **Never load-tested:** do not quote throughput numbers.

17 Interview Q&A

A. Architecture & reliability

Explain the project in one minute.

Use your 30-second pitch (section 1), then add: "the interesting parts are the fast-ack ingestion and the idempotent worker."

Why a queue? Why not insert straight into Postgres?

- Latency: the SDK runs in users' browsers; slow responses hurt them.
- Spikes: a crash storm would exhaust DB connections. The queue absorbs bursts; the worker drains at a controlled rate.
- Reliability: retries with backoff come for free.

What happens if Redis is down when an event arrives?

queue.add throws, the API returns 500 quickly (no hang). That event is lost, a known limitation. Fix: SDK retry/buffer. The rate limiter fails open so login still works.

What happens if the worker crashes mid-job?

The transaction rolls back (nothing half-written). BullMQ redelivers the job; unique event_id makes a repeat safe.

What is at-least-once delivery and how do you handle it?

A job can be delivered more than once. So the worker must be idempotent: unique `event_id`, `ON CONFLICT DO NOTHING`, counters bumped only after a successful insert, all in one transaction.

How do you stop duplicate events or double counting?

Same answer: event_id + ON CONFLICT + single transaction. If the insert conflicts, the whole attempt is a no-op (no bump, no alert).

Two workers see a new fingerprint at the same time. What happens?

Both try to insert the group; UNIQUE(project_id, fingerprint) + ON CONFLICT DO NOTHING lets one win; the other gets null and re-fetches the existing row.

Why not just catch the unique violation error?

Inside a Postgres transaction any error aborts the whole transaction; later queries fail until rollback. ON CONFLICT never raises, so the transaction stays healthy.

Why do the DB work in a transaction?

Group creation, event insert and counter updates must be atomic; otherwise a crash between steps leaves counts that disagree with events.

What do the retries look like?

3 attempts, exponential backoff 1s, 2s, 4s. After that the job sits in the failed set (kept 500) for inspection.

B. Grouping & data

How does grouping work?

Normalize the message (numbers, quoted strings, emails, URLs become *; lowercase), take the first app stack frame as funcName@fileName, SHA-256 it, keep 16 hex chars. Same hash = same group.

Why remove line numbers from the fingerprint?

They change with nearly every commit. Keeping them would split one bug into many groups after each deploy.

Why skip node_modules frames?

Library frames are identical across unrelated bugs; the first frame in YOUR code is the meaningful one.

Why truncate SHA-256 to 16 chars?

We need a short stable key, not cryptographic strength; 64 bits makes accidental collisions negligible here.

Weakness of this grouping?

Messages with genuinely different meaning but same shape can merge; the same bug via different call paths can split. Real Sentry has configurable grouping rules.

Why denormalize event_count / user_count / level?

Listing groups becomes one query with no per-row aggregate (avoids N+1). Cost: keep them in sync, which the worker transaction does.

Why is user_count recomputed, not incremented?

It is a count of DISTINCT users; the same user hitting the bug twice must not count twice.

How does PII scrubbing work and what are its limits?

Regexes for emails, cards, JWTs, password fields, long keys, plus recursive key-based redaction in context. It is heuristic, so unusual PII can slip through; user ids are only stored hashed.

Why raw SQL instead of an ORM?

Control over performance-critical queries (atomic counters, ON CONFLICT, row-comparison cursors), no hidden queries, and it shows SQL fluency. Cost: more boilerplate.

Why JSONB for breadcrumbs/context?

Variable shape, always read together with the event, never queried alone; a join table would be pure overhead.

C. Auth & security

JWT vs sessions?

JWT is stateless: any API instance verifies it without shared storage. Sessions need a store lookup per request but are easily revoked. We get both: short JWT + revocable refresh token.

Why refresh tokens, and why rotate them?

Short access token limits damage if stolen. Rotation revokes the old refresh token on each use, so a stolen one is detectable (the owner's next refresh fails).

Why store refresh tokens hashed?

Same reason as passwords: a DB leak should not hand out usable credentials.

What if a JWT is stolen?

It is valid until it expires, now only 15 minutes. The refresh token can be revoked server-side. Before this change it was 7 days with no revocation.

JWT in localStorage: risk?

Any XSS can read it. React escapes output and there is no dangerouslySetInnerHTML, which lowers risk. Alternative: httpOnly cookie plus CSRF protection.

Why does ingest use an API key instead of JWT?

SDKs run in customer apps with no logged-in user. A per-project key identifies the project and can be rotated.

Why is /api/ingest open to any CORS origin?

The SDK runs on arbitrary third-party origins by design. Protection is the API key + rate limit. Dashboard routes are restricted to CORS_ORIGIN.

Why 404 instead of 403 for someone else's project?

403 confirms the resource exists. 404 reveals nothing (prevents ID probing).

Why does forgot-password always return the same message?

Otherwise the endpoint reveals which emails are registered (enumeration).

Why does a password reset revoke all sessions?

A reset implies the account may be compromised; an attacker's existing session must not survive it.

bcrypt vs SHA-256 for passwords?

SHA-256 is fast, so brute force is cheap. bcrypt is deliberately slow, salted, and its cost factor can be raised over time.

Why bcryptjs instead of native bcrypt?

No native build in Alpine Docker. Tradeoff: slower under heavy load.

How do you protect login from brute force?

Redis-backed rate limit: 10 attempts per 15 minutes per IP, with trust proxy configured so the IP is the real client. Next step: per-account lockout.

D. Scalability & performance

Why a Redis rate limiter instead of in-memory?

In-memory counts are per process, so N replicas allow N times the limit, and the old Map never evicted keys (memory leak). Redis is shared and keys expire via TTL.

Fail open or fail closed on Redis errors?

Open: a Redis hiccup should not take login/ingestion offline. It is logged loudly. Tradeoff: briefly unthrottled.

Fixed-window weakness?

A client can send max at the end of one window and max at the start of the next (2x burst). Sliding window or token bucket fixes it.

Cursor vs offset pagination?

OFFSET scans and discards skipped rows (slower the deeper you go). Keyset jumps straight to the right place via the (project_id, last_seen, id) index.

How would you scale this to 1M events/day?

- API is stateless: add replicas behind a load balancer (rate limiter is already shared).
- Add more worker processes / raise concurrency; the queue scales consumers independently.
- DB: batch inserts, partition error_events by time, retention job, read replica for the dashboard.
- Reduce hot-row contention on popular groups by batching counter updates.

Where is the bottleneck?

Postgres writes in the worker, especially many events updating one group row (row lock serializes them).

E. Frontend, SDK, DevOps

How does the SDK avoid crashing the host app?

try/catch around the whole capture path, `.catch()` on the network call, caught serialization errors, and fire-and-forget so nothing is awaited.

Why fire-and-forget?

Reporting an error should never slow or break the app. Cost: no confirmation at the call site, acceptable for telemetry.

What are breadcrumbs for?

They show what the user did before the crash; stack shows where, breadcrumbs show how.

Why queue concurrent 401s in the axios interceptor?

Refresh tokens rotate; two parallel refresh calls would race and one would fail. Requests wait on one refresh, then retry.

How does the dashboard handle API failures?

Each fetch has a catch that shows an error with Retry, separate from the "All clear" state. (Originally missing: a real bug I fixed.)

How do you deploy / what does docker compose do?

Postgres + Redis start healthy, a one-off migrate job applies SQL, then API and worker start. Production image runs compiled JS. SIGTERM triggers graceful shutdown.

What does /health check?

Real connectivity to Postgres and Redis; returns 503 with per-dependency status when degraded.

What tests do you have and what is missing?

About 36 unit tests (fingerprint, scrubber, rate limiter) plus a smoke script. Missing: integration tests of the worker against a real DB, frontend tests, load tests.

F. Behavioral / honesty

What would you improve next?

Integration tests for the worker, real email delivery, httpOnly-cookie refresh token, per-project alerting, retention/partitioning, dashboard pagination UI.

Hardest bug you hit?

Pick from section 15: the transaction-poisoning on unique violation, or retried jobs double counting. Explain symptom, root cause, fix.

How was this built / did you use AI?

- Be honest. Say you designed the architecture and used an AI coding assistant to help write and review code, then verified it and iterated on bugs.
- Do NOT claim you hand-wrote everything. Prove ownership instead: be able to walk through fingerprint.ts, processEvent.ts, rateLimit.ts, and auth.ts line by line and change something live if asked.

Did you load test it?

No. I would not quote throughput numbers; I can describe where I expect bottlenecks and how I would measure them.

18 Resume points

Project line

- **ErrSight: Error Tracking & Monitoring Platform** | Node.js, Express, TypeScript, PostgreSQL, Redis, BullMQ, React, Docker | [GitHub link]

Bullets (pick 4-5)

- Built a Sentry-style error tracking platform: JS SDK → ingestion API → background worker → React dashboard, with Dockerized Postgres, Redis, API and worker services.
- Designed a fast-ack ingestion path (returns before any DB write) with Redis-backed per-project rate limiting and exponential-backoff retries via BullMQ.
- Implemented idempotent, transaction-safe event processing (unique event IDs, `ON CONFLICT`) so retried jobs cannot duplicate events or inflate counters.
- Built fingerprint-based error grouping (normalized messages + stable stack frames, SHA-256) and recursive PII scrubbing before storage.

- Implemented JWT access tokens with rotating hashed refresh tokens, password reset / email verification flows, and brute-force rate limiting.
- Added keyset (cursor) pagination with a matching composite index, GitHub Actions CI, and Vitest unit tests for core logic.

Do NOT write / say

- "production-ready", "highly scalable", "handles millions of events" (never load-tested).
- "fully tested", "100% coverage", "end-to-end tested" (unit tests only).
- "real-time dashboard" (it is not), "sends emails" (it logs links).
- "built entirely from scratch by hand" if AI helped; be accurate.

19 3-minute demo script

Time	What to do / say
0:00	Show the architecture diagram (README). Give the 30-second pitch.
0:20	`docker compose up --build`; show services healthy and `GET /health`.
0:40	Log in as the seeded demo user. Show the project and the API key panel (masked, copy, regenerate).
1:10	Run the 5-event test (`npm run test:ingest`): 3 identical + 2 "user 42 / user 99 not found". Point out: 5 events became 2 groups.
1:50	Open a group: occurrences, stack, context (PII shows [REDACTED]). Click Resolve; list updates without reload.
2:20	Show worker logs: new-group alert, and the "already processed, skipping duplicate" idempotency line if you replay.
2:40	Close: mention unit tests + CI, then your top limitation (email stub / no integration tests) and next step.

20 Cheat sheet & pre-interview checklist

Key numbers

- Ingest limit **100/min per project**; auth limit **10 per 15 min per IP**.
- Access token **15 min**; refresh token **30 days**; reset token **1 hour**; verification token **24 hours**.
- Caps: message **1000**, stack **5000**, breadcrumb msg **500**, breadcrumbs **20**.
- Worker concurrency **5**; retries **3**, backoff **1s → 2s → 4s**; keep **100** completed / **500** failed jobs.
- Default page size **50**, max **100**. Fingerprint = **16 hex chars** of SHA-256. bcrypt cost **10**.
- Demo login: `demo@errsight.dev` / `demopassword123` (after `npm run seed`).

Commands

```
cp .env.example .env                    # set JWT_SECRET
npm run preflight                       # checks tools, .env, packages, DB/Redis
npm run preflight -- --full             # + typecheck, tests, build (all 3 packages)
npm install && npm run typecheck && npm run typecheck:scripts
npm run test && npm run build
(cd dashboard && npm install && npm run typecheck && npm run build)
(cd sdk/javascript && npm install && npm run typecheck && npm run build)
npm install --package-lock-only         # also in dashboard/ and sdk/javascript/ (needed for npm ci / CI)
docker compose up --build
docker compose run --rm migrate npx tsx scripts/seed.ts
npm run test:ingest -- YOUR_API_KEY
cd dashboard && npm run dev
```

Before ANY interview

- **Run** `npm run preflight -- --full` **and the demo flow yourself** and see it work at least once. If any command fails, fix it first; you must not demo something you have never seen run.
- Re-read these files until you can explain every line: `services/fingerprint.ts`, `services/scrubber.ts`, `workers/processEvent.ts`, `middleware/rateLimit.ts`, `routes/auth.ts`, `dashboard/src/api/client.ts`.
- Practice the 30-second pitch and the 3-minute demo out loud.
- Memorize your limitations list (section 16) and one fix for each.
- Pick 2-3 bug stories from section 15.
- Commit the `package-lock.json` files and push to GitHub so CI shows green.